To What Extent Does Agent-generated Code Require Maintenance? An Empirical Study

Shota Sawada
National Institute of Technology, Nara College
Yamatokoriyama, Japan
AI1218@nara.kosen-ac.jp

Tatsuya Shirai
Nara Institute of Science and Technology
Ikoma, Japan
shirai.tatsuya.sp1@naist.ac.jp

Yutaro Kashiwa
Nara Institute of Science and Technology
Ikoma, Japan
yutaro.kashiwa@is.naist.jp

Ken'ichi Yamaguchi
National Institute of Technology, Nara College
Yamatokoriyama, Japan
yamaguti@info.nara-k.ac.jp

Hiroshi Iwata
National Institute of Technology, Nara College
Yamatokoriyama, Japan
iwata@info.nara-k.ac.jp

Hajimu Iida
Nara Institute of Science and Technology
Ikoma, Japan
iida@itc.naist.jp

Abstract

LLM-based autonomous coding agents have reshaped software development. While these agents excel at code generation, open questions persist about the long-term maintainability of AI-generated code. This study empirically investigates the maintenance extent, human involvement, and modification types of AI-generated files versus human-authored code. Using the AIDev dataset of AI-generated pull requests and GitHub, we analyzed over 1,000 files and approximately 3,200 changes from 100 popular repositories.

Our findings show that: (i) AI-generated files receive less frequent maintenance than human-authored code, with updates affecting only a small fraction of file size; (ii) the most frequent modifications to AI code are feature extensions, whereas human updates focus on bug fixes, and (iii) human developers perform the large majority of this maintenance.

CCS Concepts

• **Software and its engineering → Automatic programming**; *Software evolution*; *Maintaining software.*

Keywords

Agentic Coding, Software Maintenance, AI-generated code

ACM Reference Format:
Shota Sawada, Tatsuya Shirai, Yutaro Kashiwa, Ken'ichi Yamaguchi, Hiroshi Iwata, and Hajimu Iida. 2025. To What Extent Does Agent-generated Code Require Maintenance? An Empirical Study. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026).* ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

EASE 2026, Glasgow, United Kingdom

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Software maintenance is critical in the software development lifecycle to keep systems reliable, stable, and compatible with evolving technology. Developers spend considerable time on maintenance activities, including bug fixing, refactoring, and feature extensions. Previous studies [17, 23] have investigated software maintenance to understand and reduce these costs. Dehaghani *et al.* [4] report that approximately 90% of the software development life cycle is related to maintenance activities, and that these costs have increased by 50% over the past two decades.

Most of these studies, however, were conducted before the emergence of large language models (LLMs). In particular, Agentic Coding has transformed software development. Agentic Coding is an approach in which AI agents autonomously decompose high-level instructions into subtasks and perform coding activities such as writing, debugging, and refactoring with minimal human intervention. While Agentic Coding accelerates development, prior studies indicate it introduces potential risks to code quality [3, 14, 19]. He *et al.* [9] revealed that although agentic coding produces a sharp immediate increase in development velocity, it results in a 30% rise in static analysis warnings and a 41% rise in code complexity over the long term. Sankhe *et al.* [22] confirmed that while AI assistance improved productivity by 31.4%, it also led to a 23.7% increase in security vulnerabilities and a notable rise in code duplication.

These agent-related studies have primarily focused on short-term effects, such as before and after the change. The maintenance activities required after AI-generated files are introduced have received little attention, and the extent to which human intervention is required is still unclear. This study empirically investigates the maintainability of AI-generated files and the participation of AI in maintenance activities within projects that adopt autonomous coding agents. Specifically, we identify files and commits added by four major AI agents (Copilot, Claude, Devin, and Cursor) from the repositories and PR history recorded in the AIDev dataset [12], and compare them with human-generated files and commits to clarify the maintenance activities required.

Our empirical analysis of 3,238 commits shows that AI-generated files receive significantly less maintenance than human files, yet

human developers perform about 83% of it. Modifications to AI-generated files are mostly feature extensions, while human-generated files focus on bug fixes.

Replication Packages: To facilitate replication and further studies, we provide the scripts and data used in our replication package.[1]

2 Related Work

While Agentic Coding accelerates code implementation [5, 6], a growing body of work [24, 25] has begun to examine its broader effects on development tasks. Cihan *et al.* [2] reported that introducing automated code review tools increased the average time required to close pull requests. Becker *et al.* [1] conducted a controlled experiment and found that AI tool usage led to a 19% increase in task completion time.

The quality of generated code has drawn considerable attention [13]. Paul *et al.* [19] compared code smells in human-written and AI-generated code, finding that AI-generated code contains 63% more code smells on average. He *et al.* [9] reported that although development speed improves temporarily, technical debt accumulates through static analysis warnings and code complexity, leading to a decline in future velocity. From a security perspective, several studies indicate that AI-generated code is more prone to vulnerabilities than human-written code [3, 21]. Pearce *et al.* [20] evaluated code generated by GitHub Copilot against security scenarios from MITRE's "CWE Top 25"[2] and found that 40% of the generated programs contained vulnerabilities.

Maintainability issues, such as modularity and readability, have also been identified [16]. Liu *et al.* [14] investigated repetition in AI-generated code and revealed prevalent redundant repetitions at both the character and block levels, which degrade readability and efficiency. Kravchuk-Kirilyuk *et al.* [11] reported that while AI-generated code can mimic superficial modular structures, it tends to violate principles that sustain maintainability, such as encapsulation, and introduces hidden dependencies. Watanabe *et al.* [26] analyzed agent-generated pull requests and found that 9.9% of the generated methods are eventually deleted during review, placing an unnecessary cognitive burden on human reviewers.

Beyond generation-time quality, the maintenance activities surrounding AI-generated code present additional challenges. Haque *et al.* [8] reported that test code included in initial PRs authored by AI agents is often insufficient and frequently requires additional updates after the initial PR. Ottenhof *et al.* [18] conducted an empirical study on agentic refactoring and found that while human developers perform diverse structural improvements, refactorings by AI agents are dominated by superficial changes such as adding or modifying annotations. AI agents also struggle with continuous revisions during review. Minh *et al.* [15] analyzed agent-authored PRs and found that although agents excel at narrow automation, they frequently fail at iterative refinement, leading to "ghosting" (abandonment of PRs) when faced with subjective human feedback.

These prior studies primarily evaluate code quality at the point of generation or examine short-term effects immediately after AI tool adoption. However, once AI-generated files are merged into a codebase, human developers must continue to maintain them over extended periods. This long-term maintenance burden has received little empirical investigation. Our study addresses this gap by tracking the maintenance activities that follow the introduction of AI-generated files and quantifying the extent of human intervention required to sustain them.

3 Data Collection

To analyze the maintenance of AI-generated code, we need to identify files created by AI agents and track their subsequent commit histories. We used the AIDev dataset [12], which contains more than 456,000 pull requests made by five autonomous coding agents (OpenAI Codex, Devin, GitHub Copilot, Cursor, and Claude Code) from approximately 61,000 repositories. All PRs in this dataset were created between December 2024 and July 2025, following the release of agentic coding tools.

To ensure the quality of the analyzed projects, we utilized the repository list provided alongside the AIDev dataset, which consists of 2,807 repositories that have already been filtered to include only those with more than 100 stars. From this repository dataset, we identified AI-generated files through the following steps. First, we extracted files created in these PRs by identifying files marked as "added" in the Git file status of each PR. Second, we verified that each file was created by an AI agent by examining the committer name. Although the dataset guarantees that PRs are created by agents, individual commits may be made by human developers. We matched committer names against AI-agent account identifiers (*i.e.,* bots): "claude[bot]"[3] for Claude Code, "Cursor Agent"[4] for Cursor, "Copilot"[5] for GitHub Copilot, and "devin-ai-integration"[6] for Devin. We excluded Codex PRs because it does not appear as a commit owner, making it difficult to determine whether a commit was created by an agent or a developer [7]. Using this approach, we collected files generated by agents from 100 repositories. We restricted the sample to 100 repositories to reduce the high computational cost and address API limits.

Due to a large number of files from specific projects, we randomly sampled up to ten AI-generated files from each repository. For comparison, we also sampled up to ten human-generated files from the same repository created during the same period. When fewer than ten files were available in either category, we extracted all available files and balanced the dataset by selecting an equal number from the other category. For each selected file, we collected all commits recorded through January 31, 2026. This ensures a maintenance observation period of at least six months for all files because we obtained files created before July 31.

In total, we collected 508 AI-generated files and 508 human-generated files from 100 repositories, along with 1,543 commits modifying AI-generated files and 1,695 commits modifying human-generated files. We excluded the initial file-creation commits from this analysis, as we focus on subsequent maintenance activities.

[1] https://github.com/ShouSawa/AI-Code-Maintainability
[2] https://cwe.mitre.org/top25/
[3] https://github.com/claude
[4] https://github.com/apps/cursor
[5] https://docs.github.com/en/enterprise-cloud@latest/copilot/concepts/agents/coding-agent/about-coding-agent
[6] https://github.com/apps/devin-ai-integration

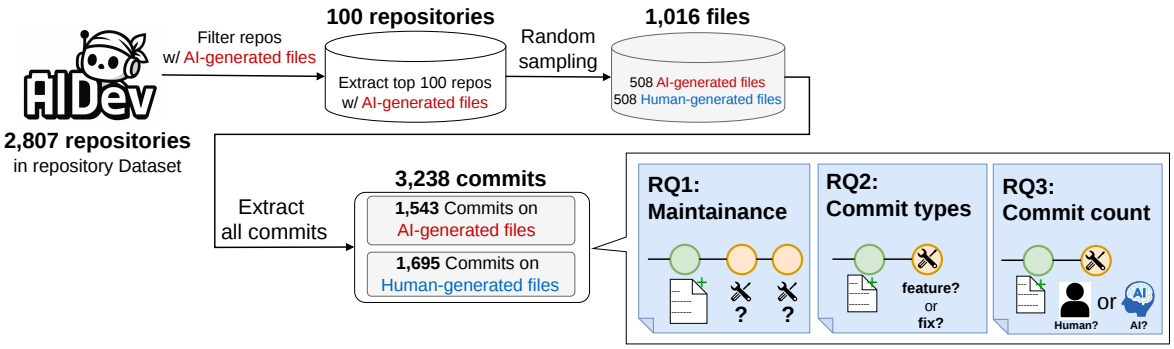


Figure 1: Overview of our data collection process and research questions

4 Research Questions

RQ_1: To what extent is AI-generated files maintained?

Motivation. Prior studies have indicated that AI-generated code tends to be lower quality compared to human-written code [3, 9, 19]. If AI-generated code demands significantly more maintenance, the productivity benefits of using AI agents may be offset by long-term maintenance costs. However, little is known about how AI-generated code is actually maintained after its initial creation.

Approach. In this study, we analyzed the extent of maintenance of AI-generated files from two perspectives: the maintenance frequency and the magnitude of the maintenance.

For the maintenance frequency, we compared the number of commits applied to AI-generated files with those applied to human-generated files to evaluate the maintenance activity associated with AI-generated code. Specifically, we aggregated commits for each file and grouped them into one-month intervals relative to the creation date. We then focused on the first six months, a period common to all files, and visualized the distributions using violin plots. We use only six months because the observation periods after file creation vary across files, making direct comparison beyond this window unreliable. Note that as we described in the Data Collection section, we ensured that at least six months have passed since the creation of all the studied files. Additionally, we did not include the first commit that created the file as maintenance.

For the magnitude, we examined whether there were differences in the number of lines of code changed per commit. However, the raw number of changed lines depends on file size, and larger files tend to have more lines modified. We normalize the changes by the size of the files to measure the percentage of changes.

Results. Figure 2a shows the number of commits for each file. The red plots represent AI-generated files, while the blue plots represent human-generated files. The horizontal axis indicates the time elapsed since file creation. To mitigate the influence of extreme values, outliers are excluded from the visualizations.

AI-generated files exhibited approximately half the commit count of human-generated files during the first month, with maintenance activity gradually declining over the subsequent three months. The distribution shows that maintenance continues at a reduced frequency rather than stopping entirely. These findings imply that AI-generated code does not impose an immediate, severe burden on developers, and that more maintenance effort is currently devoted to human-generated code.

From the fourth month onward, the gap with human-generated files narrows. The commit count stays small but non-zero, indicating that maintenance continues over the long term to a similar extent as for human-generated files. In summary, although the need for maintenance immediately after generation is lower for AI-generated files, ongoing maintenance is still required.

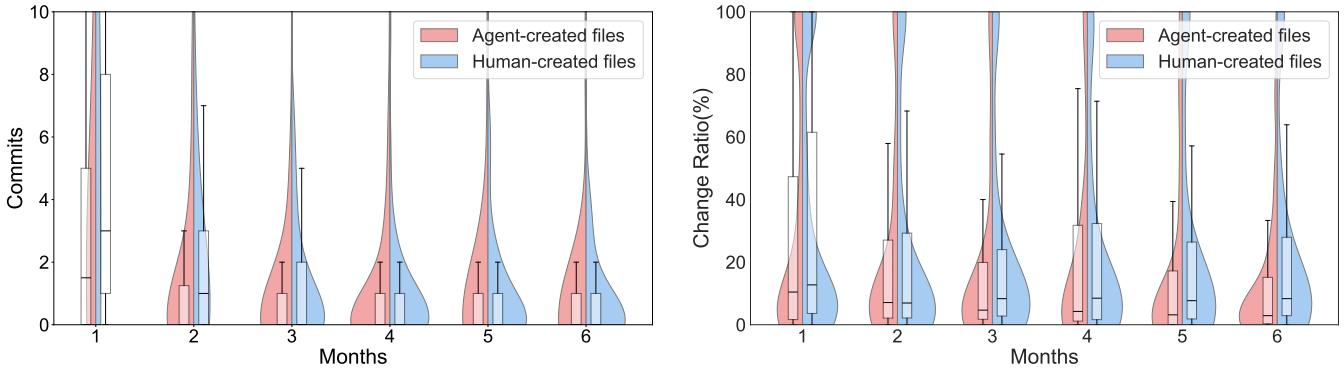
Next, Figure 2b shows the distribution of the magnitude of maintenance for each month after file creation. The results reveal that human-generated files exhibit a larger magnitude compared to AI-generated files. The smaller magnitude of changes in AI-generated files implies that maintenance is often limited to minor changes or slight modifications. Furthermore, a decreasing trend was observed in the modification ratio of AI-generated files. This indicates that the volume of maintenance decreases over time. In contrast, the higher ratio in human-generated files likely reflects more fundamental structural changes and active refactoring. Ultimately, AI-generated code does not impose an immediate, severe maintenance burden on developers. Moreover, although maintenance is required in the long term, its magnitude is smaller and less burdensome compared to human-generated files.

Answer to RQ1.

AI-generated files require less maintenance than human-generated files in both frequency and magnitude. This suggests that AI-generated code does not impose a significant maintenance burden on developers.

RQ_2: What types of maintenance activities are applied to AI-generated files?

Motivation. Watanabe *et al.* [27] show that agents perform bug fixing more frequently than feature addition. However, their study focuses only on commits within PRs generated by AI agents. It remains unclear what types of changes are made in subsequent maintenance commits after file creation. In this RQ, we examine whether modifications to AI-generated files are primarily driven by constructive activities (*e.g.*, feature additions) or by corrective actions (*e.g.*, bug fixes or refactoring).



(a) Number of commits per month after file creation

(b) Percentage of lines changed per month after file creation

Figure 2: The maintenance frequency and magnitude for Agent and Human generated files

Approach. We categorized commits modifying AI-generated files and human-generated files using the Conventional Commits Classification System (CCS) [28], an automatic classification system that defines ten commit types. CCS extends the original Conventional Commits specification[7] by providing clearer, non-overlapping definitions. This system achieves 77.95% precision and has been widely adopted in software engineering research [27].

Results. Table 1 summarizes the distribution of commit types for AI-generated files and human-generated files. For AI-generated files, the most frequent commit category was feat (21.78%), followed by refactor (14.19%), chore (13.35%), and fix (11.73%). In contrast, for human-generated files, fix was the most frequent category (16.76%), followed by docs (16.22%), refactor (15.10%), and feat (15.10%).

Compared to human-generated files, AI-generated files exhibited a lower proportion of maintenance-oriented commits such as fix and refactor, indicating fewer corrective or rework-related modifications to the generated code. Meanwhile, AI-generated files showed a notably higher proportion of feat commits, suggesting that many changes were related to functional extensions rather than corrections. In addition, fix and docs commits were more prevalent in human-generated files than in AI-generated files.

These results suggest that while AI-generated code does not frequently require bug-fixing changes, it often lacks sufficient coverage of requirements or generality, thereby necessitating additional feature implementations after the initial generation. For less frequent categories such as test, ci, and style, no substantial differences were observed between AI-generated and human-generated files, with both exhibiting similar proportions.

Answer to RQ2.

Maintenance of AI-generated files consists mainly of feature-related changes, whereas corrective and documentation-related changes are less common than in human-generated files.

[7] https://www.conventionalcommits.org/en/v1.0.0/

RQ_3: Who maintains AI-generated files: AI agents or human developers?

Motivation. As AI coding agents become more capable, they are now used not only for initial code generation but also for ongoing tasks such as bug fixes and feature additions [9, 10]. This raises an important question: *once AI agents create files, who is responsible for maintaining them?* If human developers bear this burden, they must understand and modify code they did not write. Despite the growing use of AI agents, there is limited empirical understanding of who actually performs maintenance after file creation.

Approach. For each commit modifying the studied files, we identified whether the author was an AI bot account or a human developer by matching committer names against known AI-agent account identifiers. We then calculated the proportion of commits made by AI agents and by human developers for both AI-generated files and human-generated files.

Results. We extracted 1,543 commits modifying AI-generated files and 1,695 commits modifying human-generated files. For AI-generated files, 83.21% of commits (1,284) were made by human developers, while only 16.79% (259) were made by AI agents. Similarly, for human-generated files, human developers performed 92.98% of commits (1,576), with AI agents contributing only 7.02% (119). These results indicate that maintenance is the most frequently performed by humans regardless of whether the file was originally created by an AI agent or a human developer. Notably, AI agents are more frequently involved in maintaining AI-generated files (16.79%) than human-generated files (7.02%), but human developers still handle the large majority of maintenance in both cases.

Answer to RQ3

Human developers perform the majority of maintenance for AI-generated files, while AI agents account for only a small portion of maintenance activity.

Table 1: Types of maintenance activities for Human-generated files and AI-generated files

Category	Description	Human (%)	AI (%)	Diff (%)
feat	Addition of new features	256 (15.10%)	**336 (21.78%)**	**+6.68**
build	Changes related to the build system	136 (8.02%)	155 (10.05%)	+2.03
style	Code style changes	67 (3.95%)	75 (4.86%)	+0.91
chore	Miscellaneous tasks or other changes	195 (11.50%)	206 (13.35%)	+1.85
perf	Performance improvements	38 (2.24%)	31 (2.01%)	-0.23
ci	Changes related to continuous integration (CI)	77 (4.54%)	69 (4.47%)	-0.07
test	Addition or improvement of tests	107 (6.31%)	94 (6.09%)	-0.22
refactor	Code refactoring (internal structural improvements)	256 (15.10%)	219 (14.19%)	-0.91
docs	Documentation-only changes	275 (16.22%)	166 (10.76%)	-5.46
fix	Bug fixes	**284 (16.76%)**	181 (11.73%)	**-5.03**

Note: Total commits: Human = 1,695, AI = 1,543. Percentages are calculated based on these totals. Bold numbers indicate the highest frequency in each column and the largest percentage differences. Minor categories (e.g., revert, i18n) with $< 1\%$ frequency are omitted for brevity.

Table 2: Commits to AI and human-generated files

Category	AI Commits	Human Commits	Total
AI files	259 (16.79%)	1284 (83.21%)	1543
Human files	119 (7.02%)	1576 (92.98%)	1695

5 Implication

This section discusses the implications of our findings for researchers and practitioners.

Practitioners should monitor AI-generated code beyond initial integration. AI-generated files receive less frequent maintenance and smaller magnitudes of change compared to human-authored code (RQ1). While this may suggest that AI-generated code is stable enough to function without immediate corrective action, alternative explanations are also possible; for example, developers may avoid modifying AI-generated code due to difficulty in comprehending it, or some generated files may not be actively exercised in production. Practitioners should therefore not assume that low maintenance frequency alone indicates high code quality. Moreover, subsequent maintenance predominantly involves feature extensions rather than bug fixes (RQ2), with humans still performing approximately 83% of these modifications (RQ3). This pattern indicates that even when AI-generated code operates without critical defects, it may lack full requirement coverage, necessitating human developers to build upon the initial generation. Teams adopting AI agents should establish review processes that track how generated files evolve after merging, rather than treating the initial generation as a finished product.

Researchers should develop AI agents for long-term maintenance. AI agents currently account for only about 17% of maintenance activity on the files they create. Agents are effective at initial generation but contribute minimally to the sustained maintenance lifecycle. Future research should prioritize creating agents capable of performing complex refactoring and functional growth. Investigating why AI-generated code requires more functional additions could lead to agents that better anticipate future requirements, improving the long-term utility of AI-generated software.

6 Future Direction

Extend the observation period. Our analysis covers only six months after file creation because agentic coding tools are relatively new and sufficient data points were not yet available. As these tools mature and accumulate longer histories, extending the observation window will allow us to examine whether maintenance patterns stabilize, shift, or accelerate over time. RQ1 revealed that the maintenance gap between AI-generated and human-generated files narrows from the fourth month onward, raising the question of whether AI-generated code eventually converges to or exceeds the maintenance burden of human-authored code. A longer observation period would also enable comparison across different AI agents, as each may exhibit distinct maintenance trajectories depending on its code generation strategy and the projects it is applied to.

Predict maintenance risk from code characteristics. Our current analysis focuses on the frequency and magnitude of maintenance, but does not examine what properties of the generated code correlate with higher maintenance needs. Future work should investigate whether code-level features such as cyclomatic complexity, coupling, and file size can serve as predictors of subsequent maintenance activity. For instance, identifying which combinations of these features distinguish files that stabilize quickly from those that demand sustained attention would yield actionable insights. Such a predictive model could serve as a practical tool for development teams to prioritize review effort on AI-generated files that are most likely to require future intervention.

Develop AI agents specialized in code maintenance. As shown in RQ2, the most common maintenance activity is feature extension, which requires understanding evolving project requirements and broader codebase context, tasks that go well beyond isolated code generation. Moreover, RQ3 reveals that AI agents contribute only approximately 17% of maintenance on the files they create, indicating a clear gap in their current capabilities. These findings motivate the development of maintenance-oriented agents capable of tracking requirement changes and performing targeted modifications on existing code. Combined with the risk prediction approach described above, such agents could form an end-to-end pipeline that first identifies files likely to need maintenance and then applies appropriate modifications automatically.

7 Threats to Validity

Internal validity: We categorized maintenance activities by commit type but did not control for functional complexity, which may differ between AI-generated and human-generated files. We mitigated this by randomly sampling an equal number of files from the same repositories per category, though some bias may persist. File importance and intended usage are also uncontrolled: developers may avoid AI for critical files or apply it only in specific cases.

Construct validity: The dataset covers the first six months after agentic coding tools were released, when developers are likely early adopters prioritizing feature development over maintenance. We also cannot detect AI-assisted code undeclared in commit messages, so human-generated files may include such code.

External validity: We studied only 508 AI-generated files from 100 repositories, which limits generalizability. We also excluded Codex despite its many commits. Future work should expand the dataset to address these limitations.

8 Conclusions

This empirical study examined the long-term maintainability of code produced by autonomous coding agents. Our results show that AI-generated files require less frequent modifications than human-generated files, and these modifications tend to be minor, affecting a smaller proportion of file size, with activity declining after the first month. In contrast, human-generated code receives more active structural changes throughout the observation period.

The nature of maintenance also differs: updates to AI-generated files are dominated by feature extensions (22%), whereas updates to human-generated files focus on bug fixes and documentation. Human developers perform approximately 83% of maintenance on AI-generated files, with AI agents contributing only 17%. Although the lower maintenance frequency may suggest stable code, alternative explanations such as limited usage or difficulty in comprehension cannot be ruled out.

Future research should extend the observation period beyond six months, compare maintenance patterns across different AI agents, investigate which code characteristics correlate with higher maintenance needs, and explore the development of maintenance-oriented agents that can reduce the human burden identified in this study.

Acknowledgments

We gratefully acknowledge the financial support of JSPS KAKENHI grants (JP24K02921, JP25K03100), as well as JST PRESTO grant (JPMJPR22P3), ASPIRE grant (JPMJAP2415), and CREST grant (JPMJCR23M1, JPMJCR26X7).

References

[1] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *CoRR* abs/2507.09089 (2025).

[2] Umut Cihan, Vahid Haratian, Arda Içöz, Mert Kaan Gül, Ömercan Devran, Emircan Furkan Bayendur, Baykal Mehmet Uçar, and Eray Tüzün. 2025. Automated Code Review in Practice. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2025)*. 425–436.

[3] Domenico Cotroneo, Cristina Improta, and Pietro Liguori. 2025. Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity. *CoRR* abs/2508.21634 (2025).

[4] Sayed Mehdi Hejazi Dehaghani and Nafiseh Hajrahimi. 2013. Which factors affect software projects maintenance cost more? *Acta Informatica Medica* 21, 1 (2013), 63.

[5] Sirui Hong et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *Proceedings of the twelfth International Conference on Learning Representations (ICLR 2024)*.

[6] Xingyao Wang et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR 2025)*.

[7] Github. 2025. About code owners. https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners. Accessed: December 31st, 2025.

[8] Sabrina Haque, Sarvesh Ingale, and Christoph Csallner. 2026. Do Autonomous Agents Contribute Test Code? A Study of Tests in Agentic Pull Requests. *CoRR* abs/2601.03556 (2026). https://doi.org/10.48550/ARXIV.2601.03556

[9] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2025. Does AI-Assisted Coding Deliver? A Difference-in-Differences Study of Cursor's Impact on Software Projects. *CoRR* abs/2511.04427 (2025).

[10] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. *CoRR* abs/2511.04824 (2025).

[11] Anastasiya Kravchuk-Kirilyuk, Fernanda Graciolli, and Nada Amin. 2025. The Modular Imperative: Rethinking LLMs for Maintainable Software. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages (LMPL 2025)*. 106–111.

[12] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *CoRR* abs/2507.15003 (2025).

[13] Sherlock A. Licorish, Ansh Bajpai, Chetan Arora, Fanyu Wang, and Chakkrit Tantithamthavorn. 2025. Comparing Human and LLM Generated Code: The Jury is Still Out! *CoRR* abs/2501.16857 (2025).

[14] Mingwei Liu, Juntao Li, Ying Wang, Xueying Du, Zuoyu Ou, Qiuyuan Chen, Bingxu An, Zhao Wei, Yong Xu, Fangming Zou, Xin Peng, and Yiling Lou. 2025. Code Copycat Conundrum: Demystifying Repetition in LLM-based Code Generation. *CoRR* abs/2504.12608 (2025).

[15] Dao Sy Duy Minh, Huynh Trung Kiet, Nguyen Lam Phu Quy, Pham Phu Hoa, Tran Chi Nguyen, Nguyen Dinh Ha Duong, and Truong Bao Tran. 2026. Early-Stage Prediction of Review Effort in AI-Generated Pull Requests. *CoRR* abs/2601.00753 (2026).

[16] Henrique Gomes Nunes, Eduardo Figueiredo, Larissa Rocha Soares, Sarah Nadi, Fischer Ferreira, and Geanderson E. dos Santos. 2025. Evaluating the Effectiveness of LLMs in Fixing Maintainability Issues in Real-World Projects. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER 2025)*. 669–680.

[17] Edward E. Ogheneovo. 2014. On the Relationship between Software Complexity and Maintenance Costs. *Journal of Computer and Communications* 02, 14 (2014), 1–16.

[18] Lukas Ottenhof, Daniel Penner, Abram Hindle, and Thibaud Lutellier. 2026. How do Agents Refactor: An Empirical Study. *CoRR* abs/2601.20160 (2026).

[19] Debalina Ghosh Paul, Hong Zhu, and Ian Bayley. 2025. Investigating The Smells of LLM Generated Code. *CoRR* abs/2510.03029 (2025).

[20] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *Commun. ACM* 68, 2 (2025), 96–105.

[21] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2785–2799.

[22] Purvi Sankhe, Neeta Patil, Minakshi Ghorpade, Pratibha Prasad, and Monisha Linkesh. 2025. Empirical Analysis of AI-Assisted Code Generation Tools Impact on Code Quality, Security and Developer Productivity. *International Journal For Multidisciplinary Research* (2025).

[23] Stephen R Schach. 2007. *Object-oriented and classical software engineering*. Vol. 6. McGraw-Hill New York.

[24] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. A Study on Developer Behaviors for Validating and Repairing LLM-Generated Code Using Eye Tracking and IDE Actions. *CoRR* abs/2405.16081 (2024).

[25] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Proceedings of the 2022 Conference on Human Factors in Computing Systems (CHI 2022)*. 332:1–332:7.

[26] Kan Watanabe, Tatsuya Shirai, Yutaro Kashiwa, and Hajimu Iida. 2026. What to Cut? Predicting Unnecessary Methods in Agentic Code Generation.

[27] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2025. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *CoRR* abs/2509.14745 (2025).

[28] Qunhong Zeng, Yuxia Zhang, Zhiqing Qiu, and Hui Liu. 2025. A First Look at Conventional Commits Classification. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE 2025)*. 2277–2289.